EAS 587 · SPRING 2026 · FINAL PRESENTATION

Lending Club Loan Default Prediction

Same use case, four implementations: pandas → scikit-learn → Spark + MCP → conversational UI

TEAM · 404 TEAM NOT FOUND

Aayush Nepal · Junwei Zhang · Lusi Zhang

The lender's question

> Given a loan application, what is the probability this borrower will charge off rather than fully repay the loan?

We answer it end-to-end: filter and clean the public Lending Club dataset (2014-2017), train a tuned XGBoost model with strict leakage discipline, scale the data layer to Spark + Delta Lake, and deploy two ways - **as a conversational MCP tool inside Claude Desktop** and **as this interactive app** (Predict a loan tab).

At a glance

314,212
loans in the held-out 2017 test set

21.0%
actual charge-off rate

0.726
test AUC-ROC (XGBoost, full 2017)

110
preprocessed features fed to the model

How to navigate this presentation

1. **Walk the tabs left to right** to follow the project's evolution.
2. **Predict a loan** runs the actual production model live - try the preset selector for quick scenarios, then toggle compare mode to score two loans side-by-side.
3. The **Insights + Next** tab is the team's findings + future work.
4. **Q&A** is the closing surface for questions.

Lending Club Default Risk

EAS 587 Spring 2026 - Phase 4

Active theme: *Stripe Sunset*

Team: 404 Team Not Found

- Aayush Nepal
- Junwei Zhang
- Lusi Zhang

About this app

This single Streamlit app is both the team's live demo and the visual deliverable for Phase 4. Walk through the tabs left-to-right to see the full pipeline; the **Predict a loan** tab is the live model.

Project repository

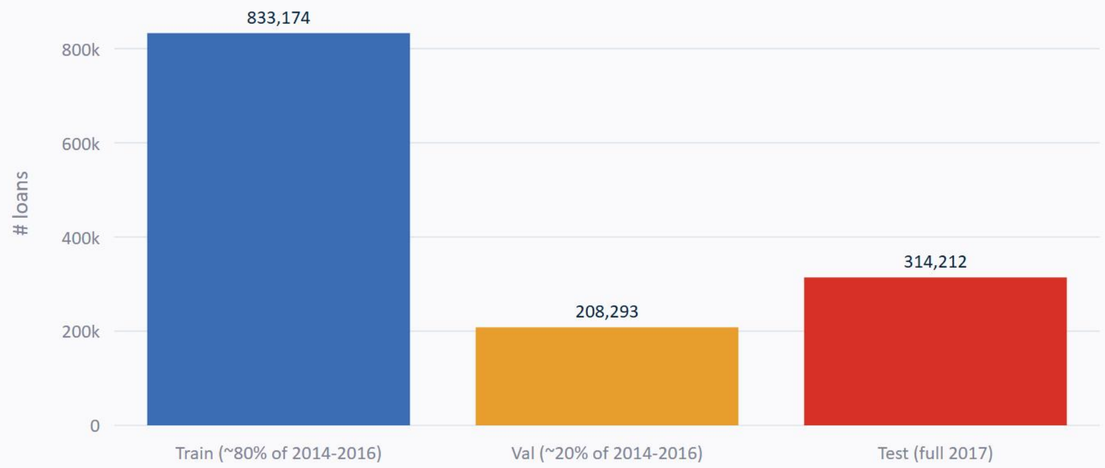
Phases 1+2 received perfect scores; Phase 3 not yet graded.

PHASE 1

Data foundations

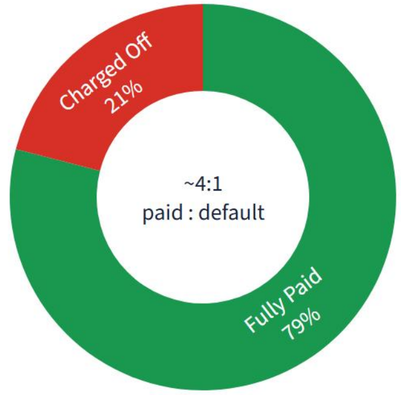
Filter, define the target, split with leakage discipline.

Time-based split (no random shuffling)



Train + val drawn from 2014-2016 by `issue_d` order; full 2017 held out as test. Random splits would leak the macro environment.

Class balance



We deliberately do NOT resample. Tree models handle the 4:1 imbalance natively via `scale_pos_weight`.

Leakage discipline - 21 post-loan columns dropped

21

post-loan columns dropped

These only get values **after** the outcome is known. Most public Lending Club notebooks accidentally train on them and report inflated metrics.

Category	Columns dropped
Payment history	`total_pymnt*` , `total_rec_prncp/int/late_fee` , `last_pymnt_*`
Outstanding balance	`out_prncp` , `out_prncp_inv`
Recovery + collection	`recoveries` , `collection_recovery_fee` , `last_credit_pull_d`
Settlement / hardship	`hardship_flag` , `debt_settlement_flag*` , `settlement_status/date/amount/percentage/term`
Updated credit signal	`last_fico_range_low` , `last_fico_range_high`

Lending Club Default Risk

EAS 587 Spring 2026 - Phase 4

Active theme: *Stripe Sunset*

Team: 404 Team Not Found

- Aayush Nepal
- Junwei Zhang
- Lusi Zhang

About this app

This single Streamlit app is both the team's live demo and the visual deliverable for Phase 4. Walk through the tabs left-to-right to see the full pipeline; the **Predict a loan** tab is the live model.

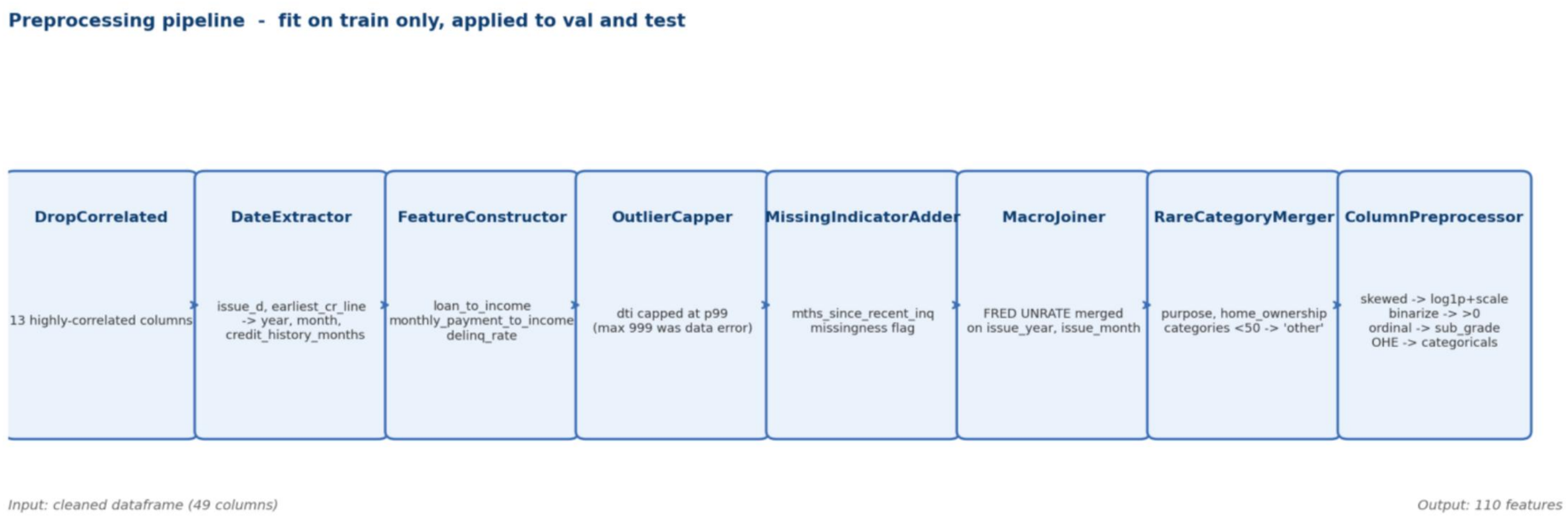
[Project repository]

Phases 1+2 received perfect scores; Phase 3 not yet graded.

PHASE 2

8-stage preprocessing pipeline

Custom scikit-learn transformers, fit on train only, applied identically to val and test.



Why each stage exists

DropCorrelated

13 highly-correlated columns removed (e.g. fico_range_high which is r=1.0 with fico_range_low).

DateExtractor

Splits issue_d and earliest_cr_line into year, month, and credit_history_months.

FeatureConstructor

Engineered features: loan_to_income, monthly_payment_to_income (PMT-formula based), delinq_rate.

OutlierCapper

dti capped at p99 - the raw max of 999 is a known data-entry error.

MissingIndicatorAdder

Some missingness is informative; we add a binary _missing flag for mths_since_recent_inq before imputing.

MacroJoiner

Pulls FRED UNRATE at fit time and joins on (issue_year, issue_month). Falls back to a 5.0% constant offline so SimpleImputer doesn't drop the column.

RareCategoryMerger

purpose and home_ownership categories with <50 occurrences fold into 'other' before OneHot.

ColumnPreprocessor

Final routing: skewed -> log1p+scale, binarize -> >0 flag, ordinal -> sub_grade encoding, OHE -> categoricals, normal -> impute+scale.

Lending Club Default Risk

EAS 587 Spring 2026 - Phase 4

Active theme: *Stripe Sunset*

Team: 404 Team Not Found

- Aayush Nepal
- Junwei Zhang
- Lusi Zhang

About this app

This single Streamlit app is both the team's live demo and the visual deliverable for Phase 4. Walk through the tabs left-to-right to see the full pipeline; the **Predict a loan** tab is the live model.

Project repository

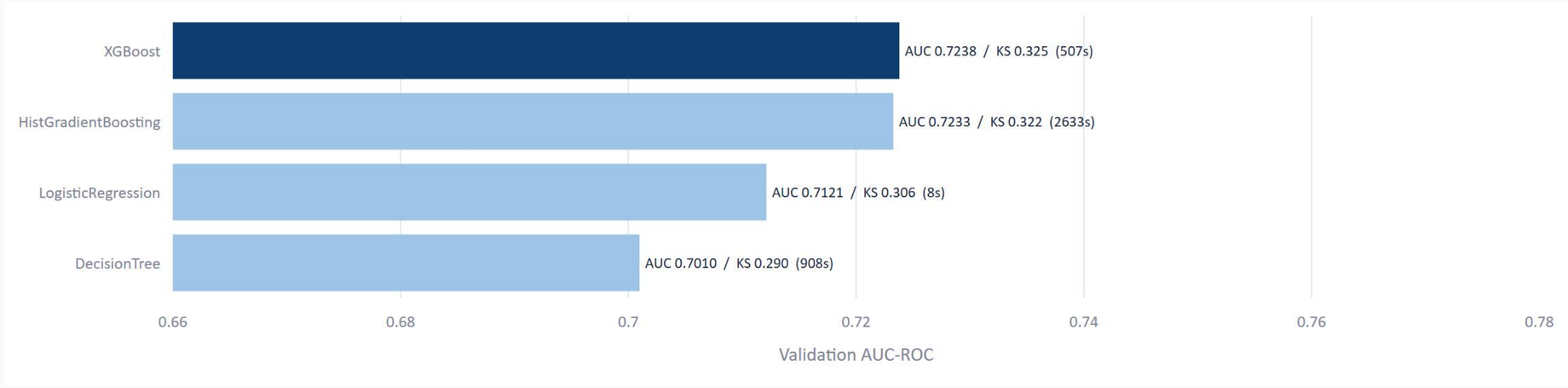
Phases 1+2 received perfect scores; Phase 3 not yet graded.

PHASE 2

Model bake-off + holdout test results

Four algorithms, 50 Optuna trials each, MLflow tracked. XGBoost won; tested on the full untouched 2017 set.

Validation set bake-off



All four share the same preprocessing pipeline above. Tuning: Optuna TPE, 50 trials per model, optimizing val AUC-ROC. All 200+ trials logged to MLflow at `models/mlruns/` . Logistic Regression is the baseline (no tuning).

Held-out test results - XGBoost on all 314,212 2017 loans

0.7260	0.4103	0.3279	67.3%
Test AUC-ROC	Test AUC-PR	Test KS	Recall on defaults (at Youden threshold 0.495)

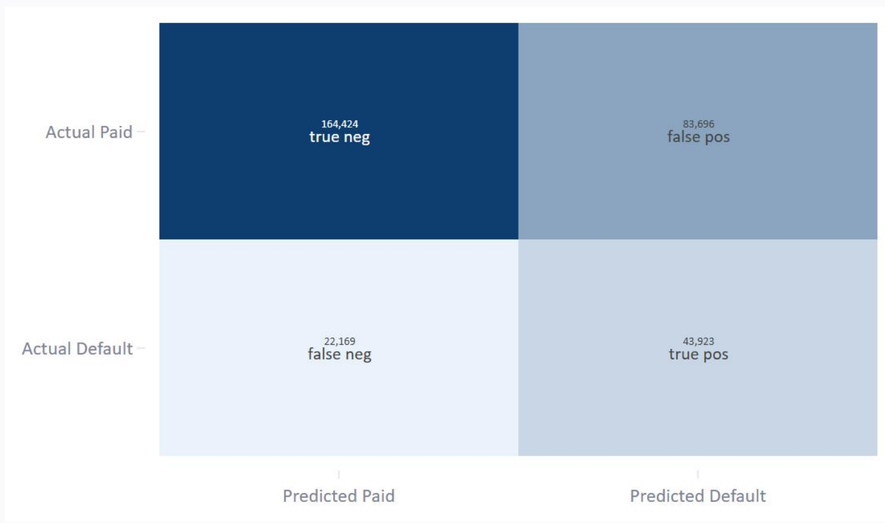
Threshold tuner - explore the operating-point trade-off live

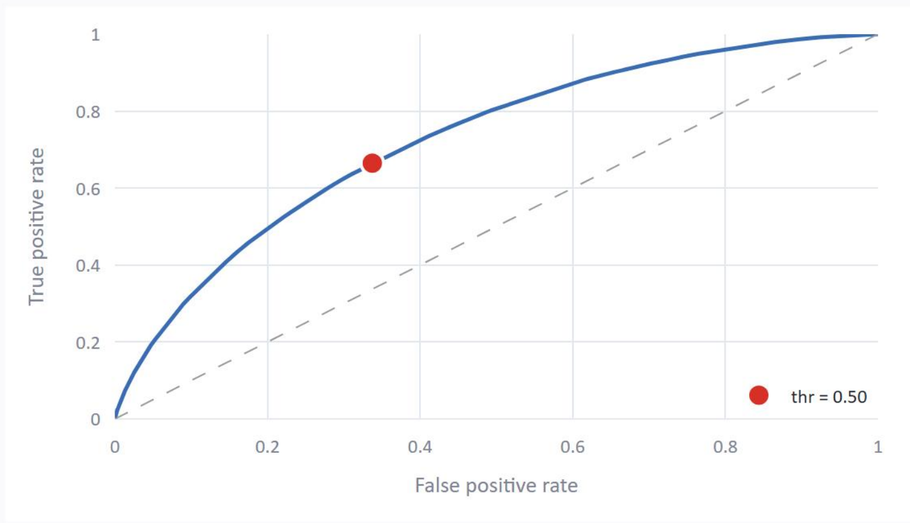
Move the slider to see precision, recall, and the confusion matrix update across the full 314k test set. Useful for stakeholders who want to choose an operating point matched to their cost ratio (e.g. a lender willing to review more loans for higher recall on defaults).

Operating threshold

0.50

Precision	Recall (defaults caught)	Accuracy	F1	% of book flagged
34.4%	66.5%	66.3%	0.453	40.6%





Also see: precision-recall curve

Lending Club Default Risk

EAS 587 Spring 2026 - Phase 4

Active theme: *Stripe Sunset*

Team: 404 Team Not Found

- Aayush Nepal
- Junwei Zhang
- Lusi Zhang

About this app

This single Streamlit app is both the team's live demo and the visual deliverable for Phase 4. Walk through the tabs left-to-right to see the full pipeline; the **Predict a loan** tab is the live model.

Project repository

Phases 1+2 received perfect scores; Phase 3 not yet graded.

PHASE 3

Spark, Delta Lake, and FRED macro overlay

Same data, same target, rebuilt on Databricks using the medallion architecture and extended with a FRED macro layer.

Medallion architecture - Bronze / Silver / Gold

BRONZE

Raw Kaggle archive landed as-is. Filtered 2014-2017 on Spark. Written to Delta as `bronze_loans`.

SILVER

Cleaned and type-fixed. Leakage columns dropped. Schema enforced. -> `silver_loans`.

GOLD

Time-based split + stratified train/val sample. Full 2017 test retained. -> `gold_loans_train/val/test`.

Secondary data: FRED macro overlay

The Phase 4 rubric specifically rewards adding recommended secondary data sources. We pulled 8 series directly from FRED (no auth) and engineered 4 more, then asked: does macro context improve the model?

National series

- `UNRATE` - civilian unemployment rate
- `FEDFUNDS` - effective federal funds rate
- `CPIAUCSL` - Consumer Price Index (all urban)
- `CPI_YOY` - 12-month CPI change (inflation proxy)

Engineered

- `real_int_rate` = `int_rate` - `CPI_YOY`
- `rate_spread` = `int_rate` - `FEDFUNDS`

State unemployment (top-5 LC volume states)

- `CAUR` , `TXUR` , `NYUR` , `FLUR` , `ILUR`
- `state_unrate` - per-borrower lookup based on `addr_state`

Production pipeline already uses macro

Phase 2's `MacroJoiner` injects `unemployment_rate` into every training row; the Phase 3 work tested whether **richer macro** (7 features) adds further lift.

Three macro insights

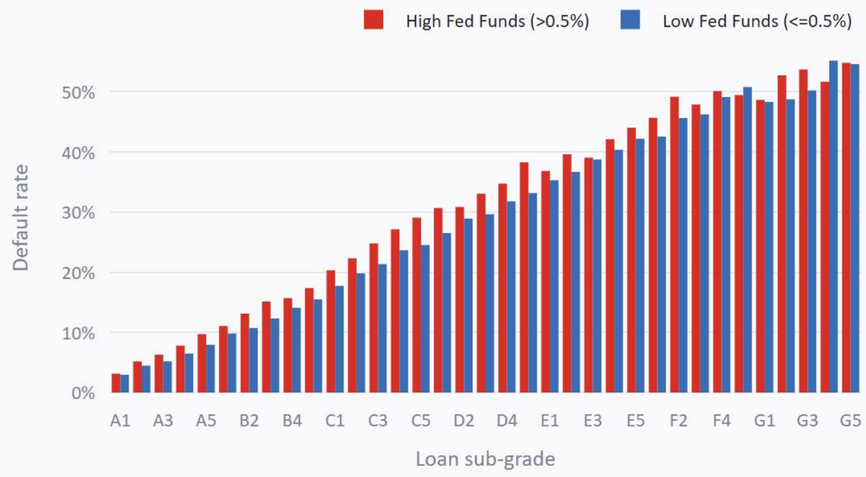
Computed locally on the 1.36M-row cleaned dataset, joined to FRED. Bars / lines reproduce the queries from notebook 05.

1. Default rate vs national unemployment at issuance



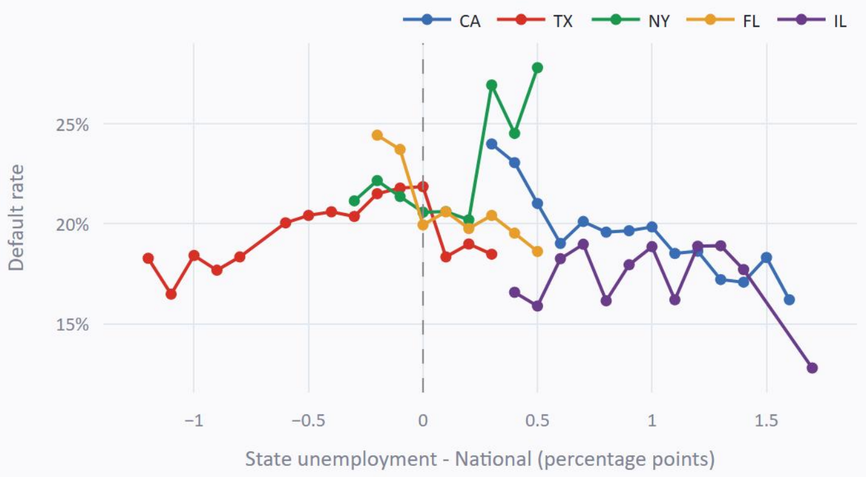
Bars = volume of loans issued at each unemployment level (left axis). Red line = charge-off rate among those loans (right axis). The relationship is positive but modest because issuance volume itself shifts with the cycle.

2. Sub-grade default rate by Fed Funds regime



The default-rate gap across sub-grades widens noticeably under higher Fed Funds rates - tighter monetary policy hits weaker borrowers harder.

3. State unemployment differential vs default rate



Among the 5 highest-volume LC states, a positive state-over-national unemployment gap correlates with higher default rates - local labor markets carry signal beyond the national series.

Honest finding: did 7 macro features improve the model?

0.7136
Phase 3 LR - loan features only

0.7131
Phase 3 LR - loan + 7 FRED features

-0.0005
Val AUC-ROC delta

Adding 7 additional FRED features beyond the production `unemployment_rate` did NOT improve LR validation AUC. The macro signal at issuance time is largely captured by the simpler single feature already in production. The macro work still earns its place: the three insights above are stakeholder-facing communication wins, and the engineered `state_unrate` and `real_int_rate` are useful for stress testing even if AUC is flat.

Lending Club Default Risk

EAS 587 Spring 2026 - Phase 4

Active theme: *Stripe Sunset*

Team: 404 Team Not Found

- Aayush Nepal
- Junwei Zhang
- Lusi Zhang

About this app

This single Streamlit app is both the team's live demo and the visual deliverable for Phase 4. Walk through the tabs left-to-right to see the full pipeline; the **Predict a loan** tab is the live model.

[Project repository]

Phases 1+2 received perfect scores; Phase 3 not yet graded.

PHASE 3

Conversational deployment via MCP

Same trained model, exposed as a Model Context Protocol tool. Claude Desktop can predict default risk from a plain-English loan description.

How it works

```
Claude Desktop
    | MCP (stdio transport)
    v
src/mcp/server.py  (FastMCP)
  - loads preprocessing_pipeline.pkl + best_model.pkl on startup
  - predict_loan_default():
      1. validates typed inputs
      2. builds a DataFrame (user inputs + 24 secondary defaults)
      3. PIPELINE.transform()  - same steps as training
      4. MODEL.predict_proba() - returns risk + tier + recommenda
```

The pickle of the preprocessing pipeline references custom transformer classes under module path `__main__` (it was pickled while the script ran as `__main__`). Both this Streamlit app and the MCP server inject those classes into `__main__` before unpickling - same trick, two consumers.

Why this matters

Most ML projects hand off a model file. Ours is reachable two ways: the visual UI on this page for analysts, and a conversational interface for everyone else. The same training-time transform pipeline runs in both consumers - guaranteed parity.

Live demo prompts (paste into Claude Desktop)

These are the exact prompts to copy-paste during the live demo. Each one demonstrates a different MCP capability.

1. Basic prediction

One loan, one tool call, one risk recommendation.

```
Predict default risk for a $12,000, 36-month loan at 11.5% intere
```

2. Multi-turn what-if (the wow moment)

Claude runs 4-5 variations through the tool autonomously.

```
How can I get it approved?
```

3. Side-by-side comparison

Claude calls the tool twice and reasons about the contrast.

```
Compare two loans: (1) $5,000 at 7%, A1 grade, FICO 780, $90K inc
```

4. Targeted what-if

Iterative tuning toward a target outcome.

```
What FICO score would this same borrower need to drop below 20% d
```

Lending Club Default Risk

EAS 587 Spring 2026 - Phase 4

Active theme: *Stripe Sunset*

Team: 404 Team Not Found

- Aayush Nepal
- Junwei Zhang
- Lusi Zhang

About this app

This single Streamlit app is both the team's live demo and the visual deliverable for Phase 4. Walk through the tabs left-to-right to see the full pipeline; the **Predict a loan** tab is the live model.

Project repository

Phases 1+2 received perfect scores; Phase 3 not yet graded.

Live model demo

Adjust borrower and loan attributes, then click Predict. After a prediction, any input change re-scores the loan live - effectively a what-if mode.

Compare two loans

Production model: **XGBClassifier**, trained on Lending Club 2014-2016 with a time-based split. 110 features after preprocessing. Scoring is identical to the MCP server in `src/mcp/server.py`.

Loan A

Quick preset: Safe (A2, FICO 760)

Loan

Loan amount ($): 8000.00
Term (months): 36
Interest rate (%): 6.50
Sub-grade: A2
Purpose: credit_card

Borrower

Annual income ($): 95000.00
DTI (%): 8.00
FICO score: 760
Home ownership: MORTGAGE
Income verification: Source Verified

Credit history

Open credit lines: 12
Revolving utilization (%): 12.00
Total credit lines (ever): 28
Delinquencies (last 2 yrs): 0
Public derogatory records: 0
Inquiries (last 6 mo): 0
Earliest credit line (YYYY-MM): 2002-05

Loan B

Quick preset: Risky (E4, FICO 620)

Loan

Loan amount ($): 32000.00
Term (months): 60
Interest rate (%): 24.50
Sub-grade: E4
Purpose: small_business

Borrower

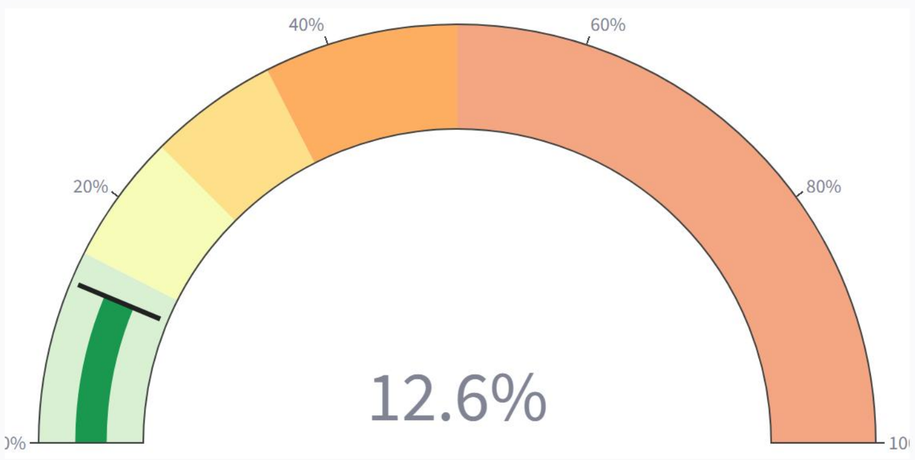
Annual income ($): 38000.00
DTI (%): 31.00
FICO score: 620
Home ownership: RENT
Income verification: Not Verified

Credit history

Open credit lines: 6
Revolving utilization (%): 88.00
Total credit lines (ever): 12
Delinquencies (last 2 yrs): 2
Public derogatory records: 1
Inquiries (last 6 mo): 4
Earliest credit line (YYYY-MM): 2012-11

Predict both loans

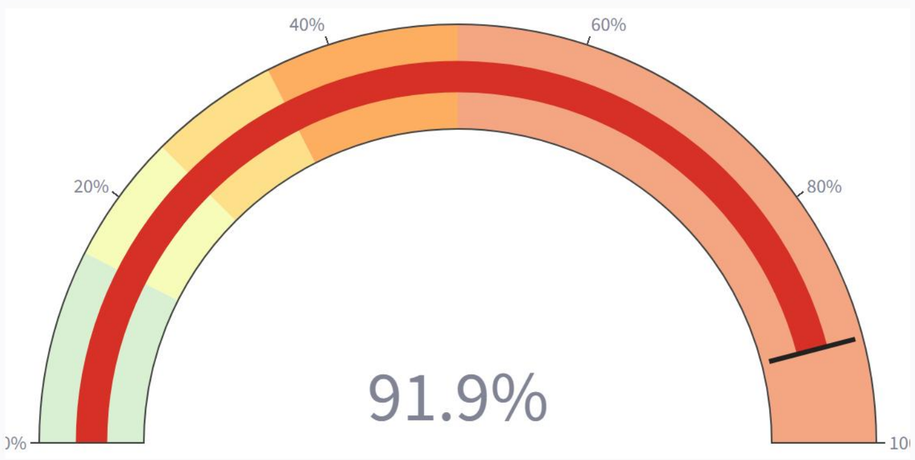
Loan A result



Very Low risk — Approve

Default probability: 12.6%
Risk tier: 1 of 5

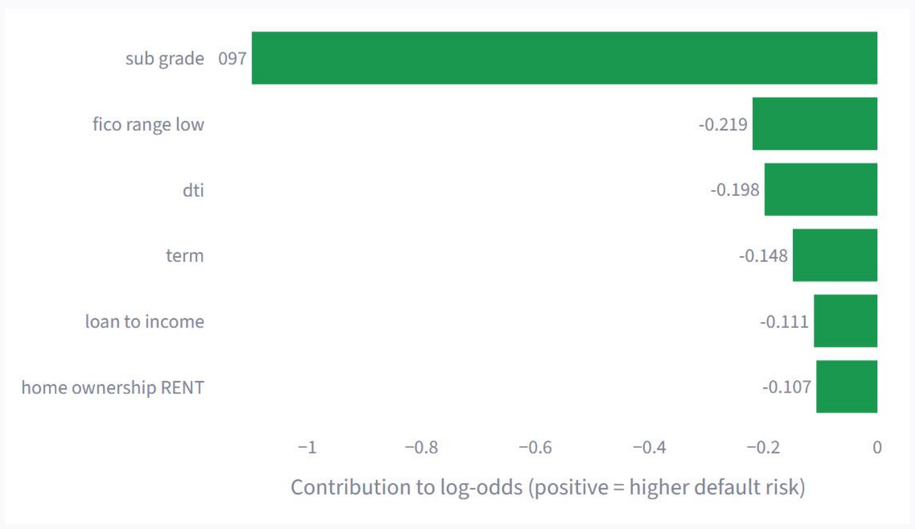
Loan B result



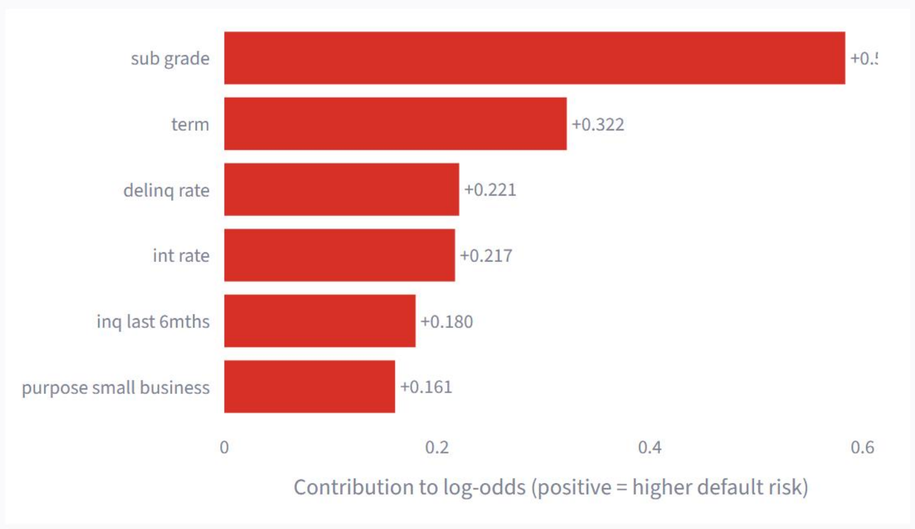
Very High risk — Decline

Default probability: 91.9%
Risk tier: 5 of 5

Why this prediction



Why this prediction



Lending Club Default Risk

EAS 587 Spring 2026 - Phase 4

Active theme: *Stripe Sunset*

Team: 404 Team Not Found

- Aayush Nepal
- Junwei Zhang
- Lusi Zhang

About this app

This single Streamlit app is both the team's live demo and the visual deliverable for Phase 4. Walk through the tabs left-to-right to see the full pipeline; the **Predict a loan** tab is the live model.

Project repository

Phases 1+2 received perfect scores; Phase 3 not yet graded.

What we found - and what's next

Key findings from the model, the three macro insights, honest limitations, and concrete future work.

Default rate ramps cleanly with sub-grade

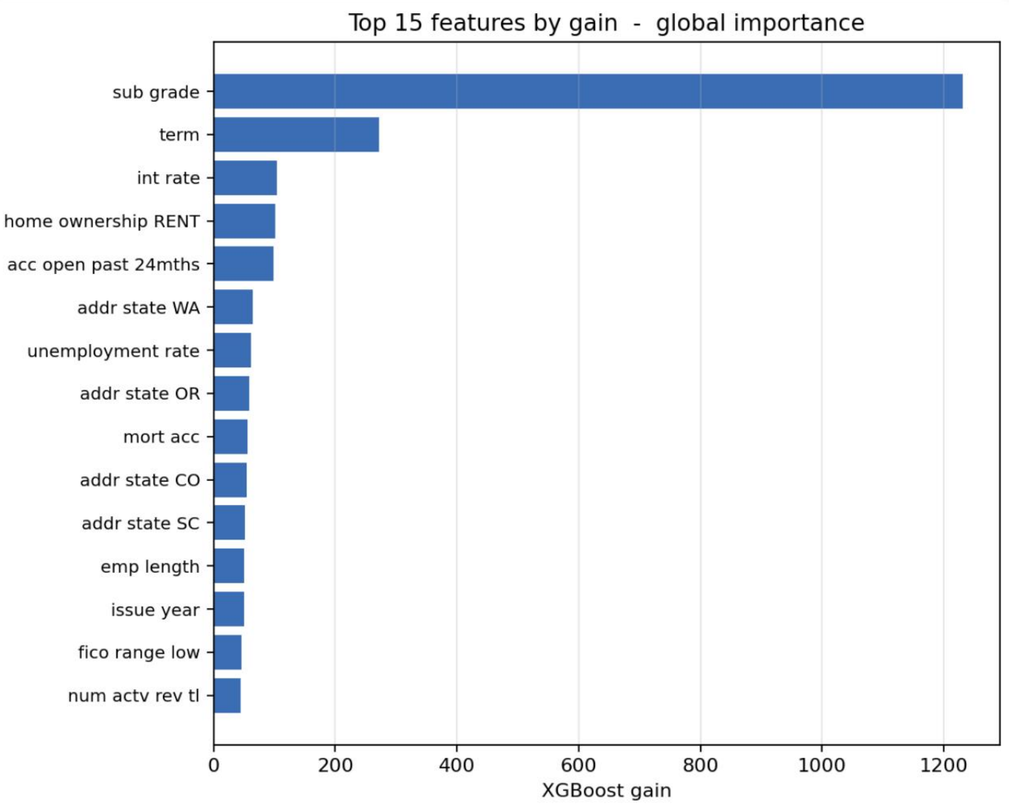


~6%
default rate at A1 (top grade)

~45%
default rate at G5 (bottom grade)

Lending Club's own sub-grade is already a near-sufficient summary of borrower risk - our pipeline mostly refines it.

What the model learned (top features by gain)



Sub-grade dominates

`sub_grade` carries roughly 3x more gain than the next feature - mirroring the chart above.

Term length is a hidden risk lever

Longer 60-month loans default substantially more often than 36-month loans at the same grade.

Renting > owning, all else equal

`home_ownership = RENT` pushes toward default; `MORTGAGE` pulls away.

Limitations and what's next

Limitations (honest)

Class imbalance left as-is

We did not resample. Any deployment with a different operating cost-ratio needs threshold re-tuning.

Concept drift after 2017

Training cohort spans falling unemployment. Model is not validated on COVID-era loan books.

No fairness audit

Lending data has well-documented protected-class disparities. Disparate-impact testing was out of scope.

FICO-only credit signal

We use `fico_range_low`; richer bureau features would likely add lift but were absent from this dataset.

What's next

Real-time scoring API

Wrap the same pipeline in a FastAPI service; the MCP server already shows the loading pattern.

Fairness + adverse-action layer

Disparate-impact metrics by protected group + per-prediction reason codes for compliant denial letters.

Retraining cadence

Quarterly re-fit on the latest closed cohort; alarm on AUC drift > 0.02.

More macro signals

FRED unemployment is one series; rates, housing, and consumer sentiment likely add lift in nonlinear models.

Use case revisited - did we solve it?

Approve / decline

The Predict-a-loan tab returns a 5-tier recommendation (Approve / Review / Decline) per borrower. The same recommendation is reachable via Claude Desktop through MCP.

Risk-based pricing

Default probability is continuous, so it directly supports tiered pricing - higher predicted risk maps to higher quoted interest rate.

Portfolio stress testing

Phase 3's macro overlay lets a portfolio manager re-score historical books under different unemployment / Fed Funds regimes - the data path exists, it's a query away.

Thank you

Questions?

404 TEAM NOT FOUND

Aayush Nepal · Junwei Zhang · Lusi Zhang

Three things to remember

WHAT WE BUILT

End-to-end loan default predictor: pandas pipeline, Spark + Delta medallion, MCP server, this Streamlit app.

HEADLINE METRIC

AUC-ROC 0.726 on 314,212 held-out 2017 loans — never seen during training or tuning.

WHY IT MATTERS

The same model is reachable two ways: a visual UI for analysts, a conversational MCP tool for everyone else.

Repository

github.com/Aayushnepal09/Eas587_project

Includes the full pipeline, the trained model artifact, the MCP server code, and the source for this app.

Likely questions, brief answers

- **Why XGBoost over HGB?** Tied on AUC-ROC at the 4th decimal; XGBoost was 5x faster to fit and exposes native SHAP contributions.
- **Why didn't the extra macro features improve the model?** `MacroJoiner` already adds `unemployment_rate`; the additional FRED features were correlated and added little marginal signal to a linear model.
- **Can this be deployed?** The MCP server is the deployment blueprint. A FastAPI version is the natural next step.